

# Informe de Avance y Documentacion del Proyecto

## Sistema Municipal de Turnos - Aplicacion del Principio de Sustitucion de Liskov (SOLID)

Fecha del informe: 17/06/2026

Repositorio fuente: [https://github.com/JoaquinCortezHub/proyecto\\_final\\_solid](https://github.com/JoaquinCortezHub/proyecto_final_solid)

Repositorio remoto local detectado: `git@github.com:JoaquinCortezHub/proyecto_final_solid.git`

### 1. Objetivo del informe

El presente informe resume el trabajo desarrollado para el Practico Nro. 3 de Programacion Avanzada, orientado a demostrar la aplicacion de un principio SOLID en un sistema real. El proyecto implementa un sistema de turnos municipales por consola, con persistencia en Supabase y separacion de responsabilidades entre modelo de dominio, menu de interaccion y acceso a datos.

### 2. Consigna tomada como base

- Aplicar correctamente un principio SOLID asignado por el docente.
- Desarrollar un sistema sobre un rubro o industria definida, en este caso gestion municipal de turnos.
- Realizar un modelo de base de datos relacional.
- Incluir diagrama Entidad-Relacion y flujo funcional.
- No es obligatorio contar con GUI, por lo que se implementa una interfaz por terminal.
- La evaluacion contempla la explicacion conceptual de los principios SOLID.

### 3. Stack tecnologico

| Elemento      | Detalle                                                                                                                         |
|---------------|---------------------------------------------------------------------------------------------------------------------------------|
| Lenguaje      | Java                                                                                                                            |
| IDE previsto  | IntelliJ IDEA                                                                                                                   |
| Interfaz      | Menu por terminal con Scanner                                                                                                   |
| Base de datos | Supabase sobre PostgreSQL                                                                                                       |
| Integracion   | Supabase REST API usando <code>java.net.http.HttpClient</code>                                                                  |
| Persistencia  | Tablas ciudadanos, servicios, turnos y pagos                                                                                    |
| Repositorio   | <a href="https://github.com/JoaquinCortezHub/proyecto_final_solid">https://github.com/JoaquinCortezHub/proyecto_final_solid</a> |

### 4. Descripcion general del sistema

El sistema permite que un ciudadano solicite turnos municipales y luego consulte sus turnos por DNI. Actualmente soporta dos tipos principales: licencia de conducir, que es un turno arancelado, y asistencia social alimentaria, que es un turno gratuito. El sistema tambien permite pagar turnos arancelados mediante la seleccion del ID del turno; en el flujo actual de demo, pagar implica eliminar el turno de Supabase.

#### Funcionalidades implementadas

- Menu principal con opciones: Sacar turno, Ver mis turnos, Pagar turno arancelado y Salir.
- Submenu para elegir entre licencia de conducir y asistencia social alimentaria.

- Asignacion automatica de fecha y hora: licencia a 7 dias a las 09:00, social a 3 dias a las 10:30.
- Reutilizacion de ciudadano existente por DNI para no pedir datos personales repetidos.
- Alta de ciudadano si el DNI no existe.
- Alta de turno asociado a ciudadano y servicio.
- Alta automatica de pago para servicios arancelados.
- Consulta formateada de turnos por DNI.
- Pago/eliminacion de turno arancelado por ID.

## 5. Aplicacion del Principio de Sustitucion de Liskov

El principio de sustitucion de Liskov establece que una clase derivada debe poder reemplazar a su tipo base sin alterar el comportamiento correcto del programa. En este proyecto, el tipo base es la interfaz **TurnoMunicipal**. Tanto **TurnoSocial** como **TurnoLicencia** pueden usarse donde el sistema espera un TurnoMunicipal, porque ambos cumplen el contrato minimo: obtener detalle, obtener fecha/hora y confirmar turno.

```
public interface TurnoMunicipal {
    String getDetalleTurno();
    String getFechaHora();
    void confirmarTurno();
}

public interface TurnoArancelado extends TurnoMunicipal {
    double calcularCosto();
    void generarBoletoPago();
}
```

El diseno evita forzar a todos los turnos a tener comportamiento de pago. Un turno social no deberia calcular costos ni generar boleto, por lo tanto no implementa TurnoArancelado. En cambio, TurnoLicencia si lo hace. Esto mantiene el contrato coherente y evita errores como UnsupportedOperationException en tipos que no deberian pagar.

### Instanciacion real en el menu

```
TurnoMunicipal turno;

if (opcion == 1) {
    turno = new TurnoLicencia(fechaHora, "B1");
} else {
    turno = new TurnoSocial(fechaHora, "Asistencia alimentaria");
}

supabaseClient.registrarTurno(
    dni, nombreCompleto, email, telefono, servicio, turno.getFechaHora()
);
```

## 6. Estructura del codigo fuente

| Archivo               | Responsabilidad                                                                        |
|-----------------------|----------------------------------------------------------------------------------------|
| Main.java             | Controla el menu de terminal, lee opciones, instancia turnos y llama a SupabaseClient. |
| TurnoMunicipal.java   | Contrato base para cualquier turno municipal.                                          |
| TurnoArancelado.java  | Contrato especifico para turnos que tienen costo y boleto de pago.                     |
| TurnoSocial.java      | Implementacion de turno gratuito.                                                      |
| TurnoLicencia.java    | Implementacion de turno arancelado para licencia de conducir.                          |
| ProcesadorTurnos.java | Servicio de apoyo para reportes y procesamiento conceptual de pagos.                   |
| SupabaseClient.java   | Capa de acceso a datos: consultas REST, altas, eliminaciones y parseo de respuestas.   |

## 7. Flujo funcional principal

Flujo para sacar un turno:

- El usuario selecciona Sacar turno.
- Elige tipo de turno: licencia de conducir o asistencia social alimentaria.
- El sistema asigna fecha y hora automaticamente.
- El sistema instancia TurnoLicencia o TurnoSocial como TurnoMunicipal.
- El usuario ingresa DNI.
- SupabaseClient consulta si el ciudadano ya existe.
- Si existe, se reutilizan sus datos; si no existe, se solicitan nombre, email y telefono.
- Se busca o crea el servicio en Supabase.
- Se crea el turno en la tabla turnos.
- Si el servicio es arancelado, se crea un registro en pagos.

Flujo para ver turnos:

- El usuario ingresa su DNI.
- SupabaseClient consulta turnos, servicios, pagos y ciudadano mediante relaciones REST.
- El menu muestra cada turno con ID, servicio, fecha, estado, tipo y datos de pago si corresponde.

Flujo para pagar un turno arancelado:

- El usuario ingresa su DNI.
- Se listan solo los turnos arancelados.
- El usuario ingresa el ID del turno.
- El sistema valida que el turno exista y sea arancelado.
- El turno se elimina de Supabase. El pago asociado se elimina por ON DELETE CASCADE.

## 8. Modelo de base de datos

| Tabla      | Campos principales                                      | Descripcion                           |
|------------|---------------------------------------------------------|---------------------------------------|
| ciudadanos | id_ciudadano, dni, nombre_completo, email, telefono     | Personas que solicitan turnos.        |
| servicios  | id_servicio, nombre_servicio, es_arancelado, costo      | Catálogo de servicios municipales.    |
| turnos     | id_turno, id_ciudadano, id_servicio, fecha_hora, estado | Registro principal de turnos.         |
| pagos      | id_pago, id_turno, monto_final, fecha_pago, estado_pago | Pagos asociados a turnos arancelados. |

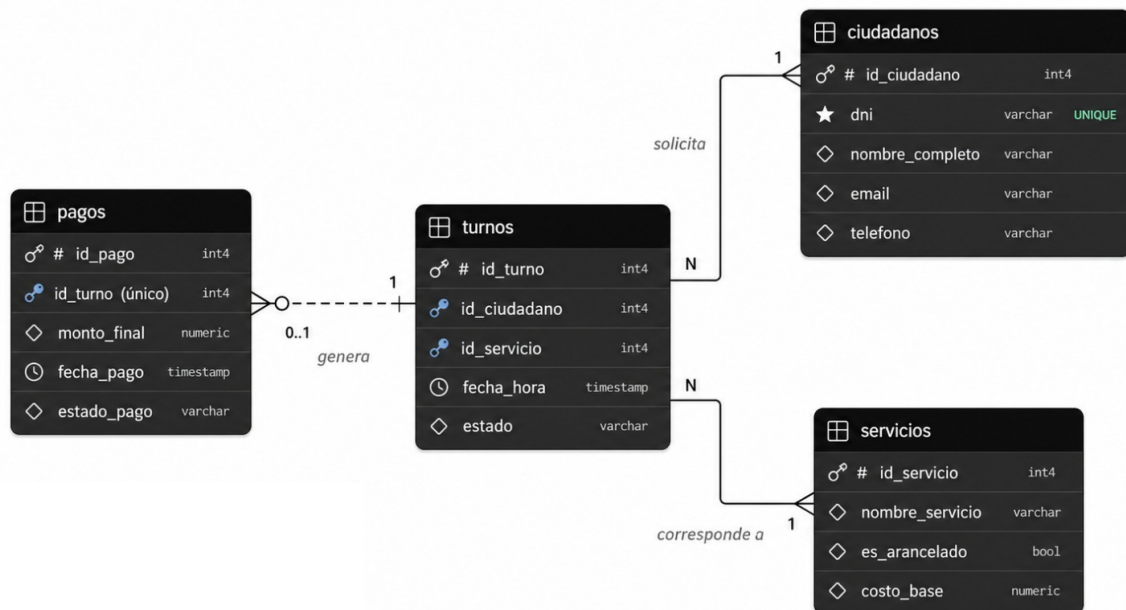
Relaciones:

- Un ciudadano puede tener muchos turnos: ciudadanos 1:N turnos.
- Un servicio puede estar asociado a muchos turnos: servicios 1:N turnos.
- Un turno arancelado puede generar un pago: turnos 1:0..1 pagos.
- La tabla pagos depende de turnos mediante id\_turno con eliminacion en cascada.

## 9. Diagrama Entidad-Relacion

Captura/diagrama incluido como evidencia visual del modelo relacional implementado en Supabase.

### Modelo Entidad-Relacion - Sistema Municipal de Turnos



## 10. Integracion con Supabase

La integracion se realiza mediante la API REST de Supabase. La clase SupabaseClient construye peticiones HTTP con HttpClient y maneja metodos GET, POST y DELETE. Para la demo local se uso una credencial de servicio en el codigo fuente; por seguridad, en este informe no se reproduce la clave completa. En una version productiva deberia moverse a variables de entorno y no subirse a GitHub.

| Operacion        | Endpoint REST                                                      | Uso                                        |
|------------------|--------------------------------------------------------------------|--------------------------------------------|
| Buscar ciudadano | /ciudadanos?select=...&dni=eq.{dni}&limit=1                        | Evita duplicar personas.                   |
| Crear ciudadano  | POST /ciudadanos                                                   | Alta si el DNI no existe.                  |
| Buscar servicio  | /servicios?select=...&nombre_servicio=eq.{nombre}                  | Obtiene ID y costo.                        |
| Crear servicio   | POST /servicios                                                    | Respaldo si la tabla servicios esta vacia. |
| Crear turno      | POST /turnos                                                       | Registra la solicitud del ciudadano.       |
| Crear pago       | POST /pagos                                                        | Se ejecuta solo para turnos arancelados.   |
| Listar turnos    | /turnos?select=...,servicios(...),pagos(...),ciudadanos!inner(dni) | Consulta formateada por DNI.               |
| Pagar/eliminar   | DELETE /turnos?id_turno=eq.{id}                                    | Elimina el turno arancelado seleccionado.  |

## 12. Estado actual y pendientes

| Area                  | Estado                                                                                                    |
|-----------------------|-----------------------------------------------------------------------------------------------------------|
| Modelo SOLID/LSP      | Implementado y demostrable con TurnoMunicipal, TurnoSocial, TurnoLicencia y TurnoArancelado.              |
| Menu terminal         | Implementado con altas, consultas y pago/eliminacion de turnos arancelados.                               |
| Supabase              | Integrado por REST API con tablas relacionales.                                                           |
| DER                   | Incluido como imagen en este informe.                                                                     |
| Seguridad             | Pendiente: mover SERVICE_ROLE_KEY a variable de entorno antes de publicar o entregar repositorio publico. |
| Videos/capturas extra | Pendiente si se desea agregar grabacion de ejecucion.                                                     |